

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Proyecto Fase 1:

Quetxal TV

PONDERACIÓN: 20 pts

Índice

Descripción del problema	3
Alcance del sistema	4
1. Autenticación, Gestión de Sesiones y Multiperfil	4
2. Gestión de Planes y Suscripciones	4
3. Catálogo, Búsqueda y Detalle de contenido	4
4. Sistema de Calificaciones Dinámico	4
5. Servicio Financiero FX-Service con Redis Cache	4
6. Historial de reproducción Reciente	4
7. Sistema de notificaciones por Correo	5
Requisitos y restricciones	5
• Desacoplamiento y Backend polígota:	5
• Código limpio:	5
• Punto de entrada único:	5
• Comunicación de Seguridad de Identidad:	5
• Programación en Base de datos:	5
• Gobierno de código:	5
• Contenedores e infraestructura:	6
• Seguridad de la información:	6
Entregables Requeridos	6
1. Requerimientos del sistema	6
2. Modelo de Casos de Uso (UML)	6
3. Vista de Arquitectura (Modelo 4+1)	6
4. Diagramas Estructurales, Comportamiento y Persistencia	6
5. Entregables de Integración y Pruebas:	7
6. Archivos de configuración:	7
Herramientas permitidas	7
Cronograma	8
Consideraciones	8

Descripción del problema

Un startup de entretenimiento digital desea lanzar al mercado una nueva plataforma de streaming de video bajo demanda. Debido a su proyección de crecimiento y la necesidad de operar en múltiples países con pasarelas de pago y divisas locales, el sistema es escalable, tolerante a fallos y desacoplado.

Para maximizar el rendimiento, reducir la latencia de comunicación y aprovechar los beneficios de diferentes lenguajes de programación, se determina el uso de que la plataforma sea construida bajo una arquitectura de Microservicios. Se deberán implementar tres lenguajes de programación simultáneos: TypeScript, Go y Python, distribuidos estratégicamente según el dominio de cada microservicio.

Toda la comunicación interna de servicio a servicio se realizará de forma síncrona y directa mediante el protocolo gRPC y se requiere el uso de contratos estrictos debido al uso de múltiples lenguajes.

La seguridad, el enrutamiento y la exposición de los servicios hacia el cliente externo se centralizará obligatoriamente a través de un API Gateway, el cual actuará como el único punto de entrada a la plataforma. El sistema debe mitigar vulnerabilidades y garantizar un control de acceso confiable mediante la integración de JWT para la propagación de identidad entre servicios, Session Cookies seguras para el mantenimiento del estado en el cliente y soporte para delegación de autorización mediante OAuth.

Asimismo, para optimizar el rendimiento de consultas complejas y delegar la lógica transaccional directamente en el motor de datos de cada microservicio, se exige el diseño de objetos de base de datos programables como procedimientos almacenados, vistas, funciones y triggers. Finalmente, con el fin de estandarizar los ciclos de desarrollo y asegurar la portabilidad de la plataforma, el aprovisionamiento de la infraestructura y el flujo de trabajo del equipo estarán normados bajo estrictas políticas de contenedores y de integración de código basadas en colaboración profesional.

Alcance del sistema

La aplicación web debe resolver de extremo a extremo los siguientes módulos y capacidades de negocio:

1. Autenticación, Gestión de Sesiones y Multiperfil

- Registro de usuarios e inicio de sesión seguro respaldado por el ecosistema de seguridad (JWT, Cookies Session y/o OAuth)
- Capacidad de crear y administrar múltiples perfiles dentro de una sola cuenta (Máximo 5 perfiles). Cada perfil debe mantener de forma aislada e independiente sus preferencias e historial de reproducción.

2. Gestión de Planes y Suscripciones

- Visualización, revisión técnica y selección de planes de suscripción (ej. Básico, Estándar, Premium).
- Panel de administración de la cuenta para modificar o cancelar la suscripción actual y actualizar credenciales de acceso

3. Catálogo, Búsqueda y Detalle de contenido

- Búsqueda avanzada y filtrado de contenido multimedia para categorías, géneros y títulos.
- Vista detallada de películas o series, incluyendo la ficha técnica y la lista de actores/reparto que participan

4. Sistema de Calificaciones Dinámico

- Los usuarios podrán calificar el contenido mediante estrellas o un sistema de recomendado (Pulgar arriba/abajo)
- La vista del catálogo debe mostrar el porcentaje global de recomendación calculando dinámicamente a partir de las calificaciones de la comunidad.

5. Servicio Financiero FX-Service con Redis Cache

- Microservicio encargado de consultar los tipos de cambio de divisas para mostrar el costo de los planes en la moneda local del usuario.
- Implementación obligatoria de una capa de caché con Redis (Con políticas TTL adecuadas) para evitar consultas repetitivas a APIs externas.

6. Historial de reproducción Reciente

- Registro del progreso de visualización por el perfil
- Para las series, se debe almacenar y mostrar la temporada, capítulo y minuto exacto donde se detuvo el usuario para permitir la reanudación inmediata del contenido

7. Sistema de notificaciones por Correo

- Módulo encargado del envío automático de correos electrónicos para confirmación de registro, recibos de compra y alertas de nuevas publicaciones de contenido.

Requisitos y restricciones

- **Desacoplamiento y Backend políglota:**

Cada dominio debe ser un microservicio totalmente autónomo con su propia base de datos independiente (patrón Database per Microservice). Los microservicios del ecosistema backend deben estar desarrollados de forma obligatoria y simultánea con los lenguajes TypeScript, Go y Python

- **Código limpio:**

Aplicación de los principios SOLID en la elaboración del código para garantizar la mantenibilidad, escalabilidad y orden en la colaboración del proyecto.

- **Punto de entrada único:**

Ningún Cliente externo puede comunicarse directamente con los microservicios del backend. El API Gateway intercepta las peticiones, validará los mecanismos de sesión y redirige el tráfico de manera segura.

- **Comunicación de Seguridad de Identidad:**

El sistema debe implementar un flujo de sesiones robusto basado en la generación de JWT para comunicación segura service-to-service, Session Cookies seguras en el cliente, y/o integración con OAuth

- **Programación en Base de datos:**

Se requiere la implementación obligatoria de:

- Procedimientos Almacenados: Para flujos transaccionales complejas (ej.. el registro unificado de un compra)
- Vistas: Para simplificar el armado de la cartelera y fichas de actores.
- Funciones: Para operaciones lógicas modulares (Ej. cálculo del porcentaje de recomendación de contenidos)
- Triggers: Para auditorías automáticas (Ej. registro de cambios de credenciales)

- **Gobierno de código:**

Los commits directos a las ramas main o develop quedan estrictamente prohibidos. Todo cambio en el código o documentación debe ser integrado obligatoriamente mediante el uso de Pull Request (PR), los cuales requerirán revisión y aprobación del equipo para simular un flujo de trabajo profesional.

- **Contenedores e infraestructura:**

- Cada microservicio, base de datos, caché y el API Gateway debe poseer su propio archivo Dockerfile para la creación de imágenes y contenedores.
- El sistema debe ser capaz de inicializar toda su topología de manera automatizada mediante Docker Compose. Se requiere la configuración obligatoria de dos entornos diferenciados:
 - Entorno local: Configurado para pruebas de desarrollo en la máquina local (docker-compose.local.yml)
 - Entorno en la Nube: Configurado con variables de entorno de producción, volúmenes remotos y políticas de reinicio adecuadas para su despliegue en un proveedor Cloud (docker-compose.cloud.yml)

- **Seguridad de la información:**

Se deben de usar de forma obligatoria archivos .env para información sensible (URLS, contraseñas, IPs, etc), este tipo de información no debe ser subida al repositorio.

Entregables Requeridos

1. Requerimientos del sistema

- Requerimientos Funcionales (RF): Listado formal, numerado y priorizado de todas las funciones de la plataforma.
- Requerimientos NO Funcionales (RNF): Especificación cuantitativa de atributos de calidad.

2. Modelo de Casos de Uso (UML)

- Diagrama de Casos de uso de alto nivel
- Primera descomposición: Diagramas detallados por módulo utilizando relaciones de inclusión y extensión.
- Casos de uso expandidos: Documentación de todos los casos de uso identificados con flujos principales, alternativos y de excepción con su propio diagrama local.

3. Vista de Arquitectura (Modelo 4+1)

- Documentación de las vistas de Escenarios, Lógica, Procesos, Componentes (Desarrollo) y Despliegue (Física) adaptadas al entorno distribuidor de microservicios, gRPC, la matriz políglota de desarrollo y configuraciones multi-entorno.

4. Diagramas Estructurales, Comportamiento y Persistencia

- Diagrama de arquitectura general, componentes y Despliegue: Deben reflejar explícitamente el API Gateway, la interconexión interna vía gRPC, la distribución de los lenguajes (Go, TS, Python) por componentes, la capa de Redis, y el mapeo físico a los contenedores Docker en ambos entornos.

- Diagrama de Actividades y Diagrama de Secuencia: Modelado temporal e interactivo de los flujos críticos (Ej. Login y validación JWT, y consumo de video)
- Diagrama Entidad-Relación (ER): Diseño de la base de datos. Debe incluir la especificación de qué componentes usarán procedimientos almacenados, vistas, funciones y triggers.

5. Aplicación

- Aplicación 100% funcional desplegada en la nube (**Obligatorio el uso de Google Cloud Platform**)
- Aplicación 100% funcional con un despliegue local.
- Código fuente de la aplicación

6. Entregables de Integración y Pruebas:

- Historial de Git e Integración (Pull Requests): Evidencia en el repositorio del uso correcto de ramas y los Pull Request aprobados para la consolidación del proyecto.
- Creación del Tag con la Versión V1.0.0.

7. Archivos de configuración:

Se deben agregar de forma obligatoria los archivos Dockerfile por servicio y los dos archivos Docker Compose utilizados para desplegar en el entorno local y en la nube.

Herramientas permitidas

Tipo	Categoría	Descripción
Obligatorio	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Obligatorio	Comunicación	gRPC
Obligatorio	Contratos	Protocol Buffers
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Obligatorio	Almacenamiento Caché	Redis
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Contenedores	Docker
Obligatorio	Orquestación	Docker-Compose

Obligatorio	Control de Versiones	Github
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io , StarUML, Visual Paradigm, FossFlow

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	01/06/2026	10/06/2026
Elaboración	01/06/2026	09/06/2026
Calificación	10/06/2026	11/06/2026

Consideraciones

- **Fecha límite de entrega:** 09 de Junio a las 23:59
- **Nombre del repositorio:** SA_PROYECTO_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Documento Técnico:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos
- No se calificará ninguna funcionalidad en el entorno local, solo se calificarán funcionalidades en el entorno de la nube.
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.
- Se calificará en base a la documentación, si algún elemento no se encuentra documentado no será tomado en cuenta para la calificación.